وزارة الاتصالات
وتكنولوجيا المعلومات



NTI
National Telecommunication Institute

iVolve

Cloud DevOps Accelerator Program

Where are we located?

5 global offices providing support round-the-clock

Our presence in 5 global regions gives us the agility to work and coordinate with you without the hassle of time zones and reachability utilizing “Follow-the-Sun” model.

Dubai	Karachi	Riyadh	San Jose	Cairo
UAE	Pakistan	Saudi Arabia	USA	Egypt



iVolve

Cloud DevOps Accelerator Hands-On Labs

- Build Tools and Unit Test Overview
- Containerization with Docker
- Container orchestration
- Continues Integration / Continues Delivery
- GitOps Workflow
- Configuration Management

iVolve

Build Tools & Unit Test Overview



iVolve

Build Tools Overview

Lab 1: Building and Packaging Java Application with Maven

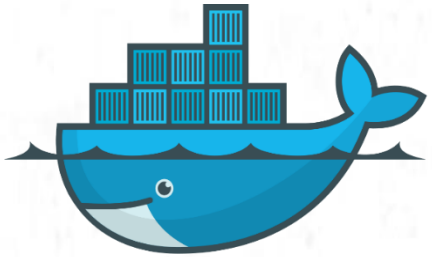
- Install java and maven.
- Clone source code https://github.com/Ibrahim-Adel15/CalculatorMavenApp.git
- Run unit test.
- Build app [generate artifact in target/calculator.jar].
- Run app.
- Verify app is working.

Lab 2: Building and Packaging Go Application with Gradle

- Install Gradle.
- Clone source code https://github.com/Ibrahim-Adel15/CalculatorGradleApp.git
- Run unit test.
- Build app [generate artifact in build/libs/calculator.jar].
- Run app.
- Verify app is working.

iVolve

Containerization with Docker



iVolve

Containerization with Docker

Lab 3: Creating a Docker Image for a Java Application

- Clone the application code: https://github.com/Ibrahim-Adel15/CalculatorMavenApp.git

Part 1: Build Inside Docker

- Write a Dockerfile using a Maven image with Java 17.
- Create a working directory and a non-root user.
- Copy the source code and build the application using mvn package
- Run the generated JAR file and expose port 8080.
- Build an image named app1 and note its size.
- Run container1, verify the application, then delete the container and image.

Part 2: Build Locally

- Build the application locally using Maven.
- Write a Dockerfile using a Java 17 runtime image.
- Create a working directory and a non-root user.
- Copy the generated JAR file into the image and expose port 8080.
- Build an image named app2 and note its size.
- Run container2, verify the application, then delete the container and image.

Bonus: Compare the image sizes and explain the differences between the two approaches.

iVolve

Containerization with Docker

Lab 4: Docker Image Optimization and Export/Load

- Clone the application code from https://github.com/Ibrahim-Adel15/ImageOptimization-.git. Build an image from the existing Dockerfile with name image1.
- Note the image size and run a container named container1 from image1.
- Verify that the application is working, then stop and delete the container.
- Write a multi-stage Dockerfile and configure the application to run as a non-root user.
- Build the optimized image as image2 and compare its size with image1.
- Run a container named container2 from image2 and verify that the application is working and running as a non-root user.
- Stop and delete container2.
- Save image2 as a TAR file named lab-image.tar.
- Delete image2 from the local Docker host.
- Load the image from image2.tar and verify that it has been restored.
- verify the loaded image is existing in the local Docker host.

iVolve

Containerization with Docker

Lab 5: Docker Image Vulnerability Scanning with Trivy

- Clone the application and Dockerfile from https://github.com/Ibrahim-Adel15/TrivyImage.git
- Build a Docker image from the provided Dockerfile and tag it as trivy-lab:v1.
- Install Trivy and verify the installation.
- Scan the Docker image for vulnerabilities and review the scan results.
- Scan the image to identify only **HIGH and CRITICAL** vulnerabilities.
- Generate an HTML vulnerability report and review the identified security issues

Bonus: Vulnerability Remediation

- Investigate the detected HIGH and CRITICAL vulnerabilities.
- Fix the identified issues by updating the vulnerable base image and application dependencies, where applicable.
- Rebuild the Docker image and tag it as ivolve-app:v2.
- Rescan the updated image and generate a new HTML report to verify the remediation.
- Ensure that no HIGH or CRITICAL vulnerabilities remain, where fixes are available.

iVolve

Containerization with Docker

Lab 6: Docker Environment Variables and Resource Management

- Clone the application code from https://github.com/Ibrahim-Adel15/PythonApp.git.
- Create a Dockerfile using the python:3.12-slim base image, install the application dependencies, and configure the application to run on port 5000.
- Build ivolve1 Docker image.
- Run a container with name container1, map port 5000, and configure the environment variables APP_NAME=iVolveApp1 and APP_ENV=stg.
- Limit the container resources to **1 CPU** and **256 MB of memory**.
- Verify that the application is working by accessing http://localhost:5000.
- Verify the environment variables configured inside the container.
- Monitor the container's CPU and memory usage using docker stats cont1.
- Create env.txt file with APP_NAME=iVolveApp2 and APP_ENV=prod.
- Run a container with name container2, map port 5000, and load the environment variables from env.txt.
- Create a new Docker Image with the following environment variables hardcoded in the Dockerfile: APP_NAME=iVolveApp3 and APP_ENV=dev.
- Run a container with name container2, map port 5000, and validate the environment variables from Dockerfile.

iVolve

Containerization with Docker

Lab 7: Multi-Network Microservices, Volumes, and Bind Mounts

- Clone the application code from https://github.com/Ibrahim-Adel15/MicroserviceApp.git.
- Create Dockerfiles for the frontend, backend, and notification services, then build their images.
- Create two bridge networks with separate subnets: frontend-backend and backend-notification.
- Create two docker volumes for backend and notification logs.
- Create host directory to persist backend application data and configure a bind mount to map it to the appropriate path inside the backend container.
- Run the notification container on backend-notification.
- Run the backend container on both networks, configure the notification service URL.
- Run the frontend container on frontend-backend, publish port 8080, and configure the backend URL.
- Access the web page and verify that messages can be created and retrieved.

iVolve

Containerization with Docker

Lab 8: Containerized Node.js and MySQL Stack Using Docker Compose

- Clone the application requirement from https://github.com/Ibrahim-Adel15/iVolveApp.git.
- The application requires a MySQL connection and a database named ivolve to start successfully.
- Create docker-compose.yml with:
  - App: Build from Dockerfile, map port 3000, configure database environment variables, and persist logs.
  - Database: Use MySQL, configure required environment variables, and persist data using db_data.
  - Network: Create a custom network named ivolve-net for service communication.
  - Volumes: Persist application logs at /app/logs/ and database data at /var/lib/mysql.
- Start the services and verify application functionality and connectivity to the DB.
- Build and tag the application image with your Docker Hub username, then push it to your repository.

iVolve

Container Orchestration



iVolve

K8S for Container Orchestration

Lab 9: Deploy Kuberneets cluster using Kubeadm

- Install and configure **containerd** as the container runtime on all nodes.
- Install **kubeadm, kubelet, and kubectl** on the control plane and worker nodes.
- Configure kubectl to communicate with the Kubernetes cluster.
- Install the **Calico CNI plugin** to enable pod networking across the cluster.
- Initialize the control plane node using kubeadm init.
- Join the worker nodes to the cluster using the kubeadm join command.
- Verify that all nodes are in the Ready state and that the cluster components are running correctly.

Lab 10: Node Isolation Using Taints and Labels in Kubernetes

- Apply a taint to a worker node with key **workload**, value **ivolveApp**, and effect **NoSchedule** to prevent pods from being scheduled on the node unless they have a matching toleration.
- Label the same worker node with **project=ivolve** to identify it for targeted pod scheduling.
- Verify that the taint and label have been applied correctly using *kubectl describe node* and *kubectl get nodes --show-labels*.

iVolve

K8S for Container Orchestration

Lab 11: Namespace Management and Resource Quota Enforcement

- Create a Kubernetes namespace named **ivolve** to isolate application resources.
- Create a ResourceQuota named **ivolveQuota** in the ivolve namespace to limit the total number of pods to **3**.
- Verify that the ResourceQuota is applied correctly and confirm that Kubernetes prevents the creation of additional pods when the limit is reached.

Lab 12: Managing Configuration and Sensitive Data with ConfigMaps and Secrets

- Create a ConfigMap named **ivolveCM** in ivolve ns to store non-sensitive MySQL configuration variables:
  - DB_HOST: The hostname of the MySQL StatefulSet's headless Service.
  - DB_USER: The application database user authorized to connect to the ivolve database.
- Create a Secret named **ivolveSecret** in ivolve ns to store sensitive MySQL credentials:
  - MYSQL_ROOT_PASSWORD: The root password for the MySQL database.
  - DB_PASSWORD: The password for the application database user specified by DB_USER.
- Use **Base64 encoding** to encode the Secret data values when defining the Secret using a YAML manifest.

iVolve

K8S for Container Orchestration

Lab 13: Persistent Storage Setup for Application Logging

- Define a Persistent Volume (PV) with the following specifications:
  - **Name:** ivolvePV
  - **Size:** 1Gi
  - **Storage type:** hostPath
  - **Path:** /mnt/app-logs on the node file system
  - **Access mode:** ReadWriteMany (to allow all replicas read/write access)
  - **Reclaim policy:** Retain
- Define a Persistent Volume Claim (PVC) with name **ivolvePVC** that requests 1Gi storage.
- Ensure the PVC access mode matches the PV (ReadWriteMany).
- Do not use any StorageClass.

Lab 14: StatefulSet with Headless Service

- Create **ivolveDB** StatefulSet in ivolve ns with 1 replica running MySQL.
- Configure the StatefulSet pod to consume environment variables from the ConfigMap and Secret.
- Add node selector to ensure the pod will run on node with label **'project=ivolve'**.
- Add a toleration to the StatefulSet with key-value **'workload=ivolveApp'** and effect **NoSchedule**.
- Configure a Persistent Volume Claim (PVC) template to mount /var/lib/mysql in the pod.
- Write a YAML manifest for a headless service with name **ivolveDBSvc** targeting the MySQL StatefulSet pod.
- Confirm the database is operational by connecting with a MySQL client.
- Verify Mysql pod DNS resolution inside the cluster.

iVolve

K8S for Container Orchestration

Lab 15: Node.js Application Deployment with ClusterIP service

- Create a Deployment named **ivolveApp** in ivolve ns with 2 replicas.
- Use your custom Docker image from Docker Hub created in Lab 8.
- Configure the pods to consume environment variables from the existing ConfigMap and Secret.
- Add a node selector to schedule pods on nodes labeled project=ivolve.
- Add a toleration with key workload, value ivolveApp, and effect NoSchedule.
- Configure the pods to use the PersistentVolume created in Lab 13.
- Add an init container using a MySQL client image to create the ivolve database and grant the application user the necessary privileges.
- Pass the database connection parameters to the init container using environment variables from the ConfigMap and Secret.
- Create a ClusterIP Service named **ivolvaAppSvc** in ivolve ns to distribute traffic across the available application pods.
- Connect to MySQL manually to verify that the ivolve database was created and the application user has the required privileges.
- Check the application logs to verify that the Node.js application successfully connects to MySQL and accesses the ivolve database.

K8S for Container Orchestration

Lab 16: Pod Resource Management with CPU and Memory Requests and Limits

- Update the existing **ivolveApp** Deployment to include resource requests and limits.
- Resource Requests:
  - ✓ cpu: 1 (1 vCPU)
  - ✓ memory: 1Gi
- Resource Limits:
  - ✓ cpu: 2 (2 vCPUs)
  - ✓ memory: 2Gi
- Use kubectl describe pod to verify the applied resource requests and limits.
- Use kubectl top pod to monitor real-time resource usage (metric server must be running in the cluster).

Lab 17: Control Pod-to-Pod Traffic via Network Policy

- Define a Network Policy with the following specifications:
  - Name: ivolveNP
  - Pod Selector: Targets mysql pod.
  - Policy Types: Only Ingress
  - Ingress Rule:
    - ✓ Allows traffic only from the application pods .
    - ✓ Restrict access to port 3306 (MySQL default port).

iVolve

K8S for Container Orchestration

Lab 18: Node-Wide Pod Management with DaemonSet

- Create **monitoring** namespace.
- Deploy **Prometheus** in the monitoring namespace.
- Deploy **Alert Manager** in the monitoring namespace.
- Deploy **Grafana** in the monitoring namespace.
- Deploy a DaemonSet for Prometheus **node-exporter** in the monitoring namespace.
- Validate that a node-exporter pod is scheduled on each node.
- Confirm correct metrics exposure by accessing <node_IP>:9100/metrics.
- Configure Prometheus to collect data from node-exporter.
- Configure Grafana dashboard for Kubernetes.

Lab 19: Securing Kubernetes with RBAC and Service Accounts

- Create **jenkinsSA** Service Account in the ivolve namespace.
- Create a **token** for the service account.
- Define a Role named **ivolveDeployRole** that grants create/update permissions on deployment in the ivolve namespace.
- Create a Role Binding with name **ivolveDeployRoleBinding** to bind the ivolveDeployRole role to the jenkinsSA service account.
- Validate the role can only create/update deployment in ivolve namespace.

iVolve

Continues Integration / Continues Delivery



iVolve

Continues Integration / Continues Delivery

Lab 20: Role-based Authorization

- Create user1 and user2 in Jenkins.
- Assign admin role for user1 & read-only role for user2.

Lab 21: Jenkins Pipeline for Application Deployment

- Clone source code and Dockerfile from: https://github.com/Ibrahim-Adel15/DemoApp.git
- Create a pipeline that automates the following processes:
  1. Build app
  2. Build Docker Image.
  3. Scan Docker Image
  4. Push image to DockerHub.
  5. Delete image locally.
  6. Update image in deployment file.
  7. Deploy to Kubernetes cluster.
- Set pipeline post action (always, success, failure)

iVolve

Continues Integration / Continues Delivery

Lab 22: CI/CD Pipeline Implementation with Jenkins Agents and Shared Libraries

- Clone Dockerfile from: https://github.com/Ibrahim-Adel15/DemoApp.git
- Create the following shared libraries:
  1. **BuildApp** (Unit test, Build app)
  2. **BuildImage** (Build image, scan image push image, delete image locally)
  3. **DeployOnK8s** (edit deployment file, deploy to cluster)
- Update your pipeline to use them.
- Configure Jenkins slave to run the pipeline.

iVolve

GitOps Workflow



iVolve

Continues Integration / Continues Delivery

Lab 23: Implementing a GitOps Workflow with Jenkins and Argo CD

- Install Argo CD in your Kubernetes cluster.
- Access the Argo CD UI and log in successfully.
- Connect Argo CD to your GitHub repository.
- Create an Argo CD Application named ivovle.
- Configure the application to monitor the appropriate GitHub repository and manifest directory.
- Configure automatic synchronization so that Argo CD automatically applies changes detected in Git.
- Modify the **DeployOnK8s** Jenkins role to Push new update files into Github repo after edit the deployment file.
- Trigger the Jenkins pipeline to build and publish a new application image.
- Confirm that Argo CD detects the changes and synchronizes the application.

iVolve

Configuration Managment



iVolve

Red Hat Enterprise Linux Automation with Ansible (RH294)

Lab 24: Initial Ansible Configuration and Ad-Hoc Execution

- Install and configure Ansible on control node.
- Generate new key on control node.
- Transfer the public key into managed node using ssh-copy-id command.
- Create inventory of a managed node.
- Perform ad-hoc command to install Docker.

Lab 25: Automated Web Server Configuration Using Ansible Playbooks

- Write an Ansible playbook to automate the following tasks:
  - ✓ Install Docker
  - ✓ Install Kubectl
  - ✓ Install Jenkins
- Verify the configuration on managed node.

Lab 26: Structured Configuration Management with Ansible Roles

- Create separate Ansible roles for Docker, kubectl, and Jenkins.
- Write Ansible playbook to run the created roles.

iVolve

Red Hat Enterprise Linux Automation with Ansible (RH294)

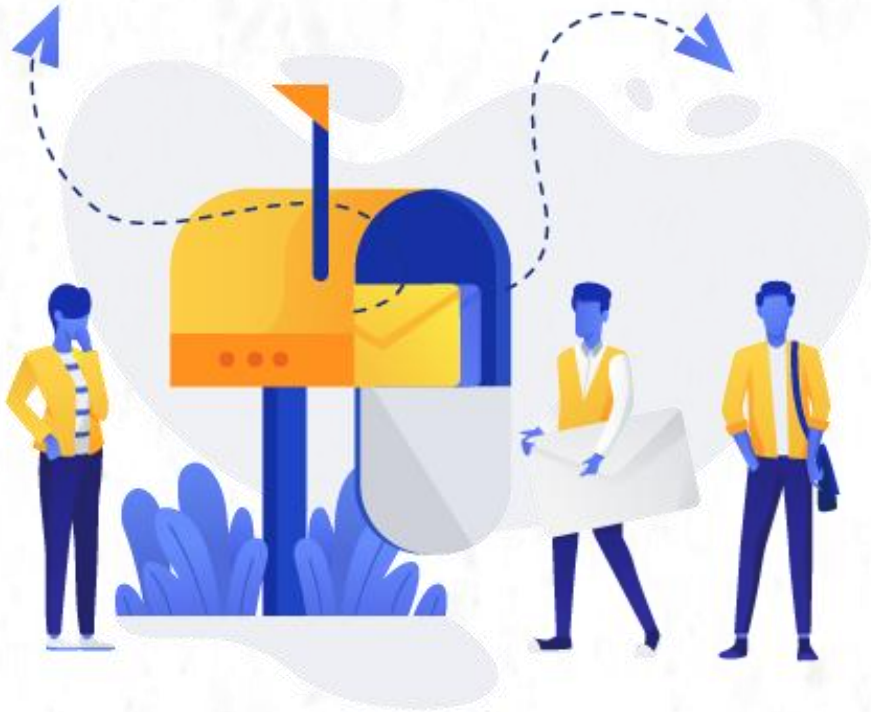
Lab 27: Securing Sensitive Data with Ansible Vault

- Create MySQL Role to automate the following tasks:
  - ✓ Install MySQL.
  - ✓ Create ivovle database.
  - ✓ Create ivolveuser user with all privileges on ivolve DB.
- Use Ansible Vault to encrypt sensitive information such as database user password.
- Validate DB on managed node by connect to database using the created user and listing databases.

Lab 28: Automated Host Discovery with Ansible Dynamic Inventory

- Create AWS EC2 with tag server:ivolve.
- Set up Ansible dynamic inventory to automatically discover and manage running EC2 with tage server:ivolve.
- list the target hosts using ansible inventory command.
- Run a playbook on this EC2.
- Verify the configuration on the EC2.

iVolve



Your partner in the cloud.

Have Any Questions?
Feel Free to Contact Us.

SAUDI ARABIA

18th Floor, Faisaliah Tower,
Olaya Street, Riyadh, 12212
Kingdom of Saudi Arabia

Egypt

Westown Hub Building 3, Sodic, Beverly Hills, Cairo-Alexandria Desert Road, Unit C41

USA

123 E San Carlos St. PMB 4575 San Jose, CA 95112, USA

UAE

Dubai, Silicon Oasis, Techno Hub. 01,
Technology Entrepreneurship Center (DTEC)
P.O. Box: 6009, Dubai, UAE

Pakistan

NGC Tower, 3rd Floor, Plot # A-1, Block-3 7/8
KECHS, Jinnah Housing Society, PECHS, Karachi, 75000

> sales@ivolve.io

> info@ivolve.io

Talk To Our Representatives

iVolve

THANK YOU!

Evolve your CLOUD&DIGITAL Transformation journey

with

